

ELI — Marketing and Outreach Playbook

Contents

0. Read this first — mindset and realistic expectations	1
1. Pre-launch checklist — get the repo right BEFORE driving traffic	1
2. Accounts you'll need — and the "new account" trap	2
3. The launch sequence — a two-week calendar	2
4. Channel-by-channel playbook	3
5. Engagement playbook — handling the comments	4
6. The demo clip — your single most important asset	5
7. Metrics and realistic expectations	5
8. Do's and don'ts (anti-spam survival guide)	5
Appendix A — Copy-paste drafts	6
Appendix B — The 60-second pre-flight checklist (before you click "post")	8

One thing to set before you use this: wherever you see **[YOUR MODEL]**, replace it with the model you actually daily-drive (e.g. *Qwen2.5-7B*). It appears in the post drafts.

0. Read this first — mindset and realistic expectations

Before any tactics, three honest truths so you spend your energy where it actually pays off:

1. **Honesty wins; hype loses.** Every community below is sharp and has seen a thousand over-promised "AI assistant" posts. Your strongest move is candour: say what ELI is, what it genuinely does, and where it falls short. That earns trust and upvotes. Marketing-speak gets downvoted and picked apart.
2. **Notice is achievable; donations rarely are.** One good post in the right place can take you from zero to thousands of views in a day. Money is a different story — most projects, even popular ones, earn little or nothing. Set the donation jar out, expect nothing, treat anything that comes as a nice surprise. The real, realistic win is **a handful of people who genuinely get it, use it, and give you feedback.**
3. **One great post beats ten mediocre ones.** Do not blast every forum on the same day — that reads as spam and gets you filtered or banned. This playbook is **sequenced** on purpose.

What a good outcome looks like (so you can judge it fairly): a few hundred to a few thousand repo visitors over a launch week, tens to low-hundreds of stars, a dozen genuine comments, a few people who try it, and one or two useful bug reports or ideas. That is a *success* for a solo project. Anything beyond it is a bonus, not the baseline.

1. Pre-launch checklist — get the repo right BEFORE driving traffic

Your GitHub page **is** your landing page. Sending people to a weak page wastes the one shot a post gives you. Tick all of these first.

Item	Why it matters	Status
Clear one-line description + README first screen	People decide in ~10 seconds	Done
Repo topics/tags	Surfaces you in GitHub search & topic browse	Done (12 tags)
License stated clearly	Avoids confusion & license fights	Done
Demo GIF/video at the top of the README	The single biggest lever — see §6	To do
Screenshots in the README	Shows it's real and looks finished	Recommended
Social-preview image	The picture shown when your link is shared anywhere	To do (§4.5)
A tagged Release (e.g. v2.0)	Signals “maintained”, gives a download point	To do (§4.5)
Install steps that work from a clean clone	First thing a curious dev tries	Verify
Sponsor button / Ko-fi live	So interested people can support	Done

Rule of thumb: do not post anywhere until the **demo clip** and **social-preview image** are done. They more than double the return on every post.

2. Accounts you'll need — and the “new account” trap

Most platforms **silently filter brand-new, zero-history accounts** as suspected spam. If you make an account today and immediately post a link to your own project, there is a real chance **no one ever sees it** (it goes to a mod queue or gets shadow-filtered). Avoid this by *warming up* first.

- **Reddit** (your primary channel):
 1. Create an account a few days **before** you plan to post.
 2. “Warm it up”: genuinely comment on a handful of posts in r/LocalLLaMA and related subs over 2–3 days. Aim for ~20–50 comment karma. This is not gaming — it's being a real participant.
 3. Check each subreddit's **minimum karma / account-age** rule (in the sidebar/rules). Some require a few days and a little karma to post.
- **Hacker News**: create an account now (a little account age helps credibility). You don't need karma to submit, but lurk a bit so you know the tone.
- **Optional, later**: X/Twitter, Mastodon (a fediverse server), dev.to, YouTube. Make these only if you'll actually use them.
- **Lobsters** (lobste.rs): invite-only — skip unless someone invites you.

Never create an account and immediately drop five self-links. That's the fastest way to get auto-banned and accomplish nothing.

3. The launch sequence — a two-week calendar

Sequenced so each step builds, you're never overwhelmed, and you never look like a spammer. Spread it out; reply to comments between steps.

When	Action	Notes
Week 0 (prep)	Finish §1 assets; warm up Reddit + make HN account	Don't skip the demo clip
Day 1 (Tue–Thu, 8–10am US ET)	Post to r/LocalLLaMA (your best shot)	Full draft in Appendix A2

When	Action	Notes
Day 1-2	Reply to every comment within minutes-hours	Early engagement ranks the post
Day 3-4	Hacker News “Show HN” (if you have energy)	Appendix A3; expect bluntness
Day 5-7	r/selfhosted (privacy/self-host angle)	Appendix A4 — reframed, not copy-paste
Week 2	r/Python (technical angle), a short YouTube demo, one X/Mastodon post, optional dev.to writeup	Appendix A5-A7
Ongoing	Iterate the README from feedback; later post “now it does X” updates	Updates are fair game on HN/Reddit

Why spread out: posting the same link to many places in one day is the classic spam pattern. One strong post per channel, days apart, with you present to engage, is what actually works.

4. Channel-by-channel playbook

4.1 r/LocalLLaMA — your #1 priority

- **What it is / who’s there:** the main Reddit community for running LLMs on your own hardware — hobbyists, engineers, tinkerers. Your *exact* audience: people who already want a private, local assistant.
- **Read the rules first.** Open the subreddit, read the sidebar + rules. Note any **self-promotion policy**, required **post flair**, and karma/age minimums.
- **Step by step:**
 1. Make sure your account is warmed up (\$2) and the demo clip is ready (\$6).
 2. Click “Create Post” → choose the subreddit.
 3. **Title:** use Appendix A2’s title (clear, no hype).
 4. **Body:** paste Appendix A2’s body. Keep paragraphs short.
 5. **Attach the demo clip** (upload the MP4/GIF directly, or link it near the top of the body).
 6. **Pick flair** if prompted (e.g. “Resources” / “Other”).
 7. **Post Tuesday-Thursday, ~8-10am US Eastern** for the most eyes.
 8. **Reply to every comment** in the first 1-2 hours — this is what pushes it up the page.
- **Pitfalls:** hype words, getting defensive about the license, posting at a dead hour, dropping the link and disappearing.

4.2 Hacker News — “Show HN”

- **What it is / who’s there:** a programmer-heavy news site. Bursty traffic; a hit sends a real wave. Crowd is technical, blunt, and very pro-open-source.
- **Mechanics (different from Reddit):** you submit the **repo URL as the link**, then **immediately post your explanation as your own first comment** (Appendix A3). Submit at news.ycombinator.com/submit. Title must start with **“Show HN:”**.
- **Timing:** weekday, ~8-10am US Eastern. A quiet hour can bury it before anyone sees it. (HN also has a “second-chance” pool that re-surfaces some posts; don’t repost manually if it flops.)
- **Behaviour:** expect direct, sometimes harsh, technical questions and **license pushback** (“why not OSI open-source?”). Answer calmly and factually — engaged, non-defensive authors do well. **Never ask for upvotes** (against the rules, and it backfires).

4.3 r/selfhosted

- **Angle:** privacy and owning your own stack — *not* the same pitch as r/LocalLLaMA. Lead with “runs entirely on your machine, no cloud, offline-enforced.” Use the reframed draft (Appendix A4).
- **Rules:** check the self-promo policy; some self-host subs want you to be an active participant first. Post a few days after r/LocalLLaMA, not the same day.

4.4 Other subreddits — use with care

Subreddit	Worth it?	Note
r/Python	Yes (week 2)	Lead with the engineering: 140k LOC, pipeline, agents. Honest “Show & Tell”.
r/artificial	Maybe	Broader, less technical; lower-quality engagement.
r/opensource	Be careful	It’s source-available, <i>not</i> OSI open-source — some there will object. Be upfront or skip.
r/MachineLearning	Probably skip	Strict, research-focused; a product post may not fit.
r/LocalLLM, r/Oobabooga, r/ollama	Optional	Smaller, on-topic; fine later, spaced out.

4.5 GitHub itself (free, high-leverage)

- **Social-preview image:** Settings → General → “Social preview” → upload a simple 1280×640 image (logo + one line: “ELI — a fully-local AI assistant”). This is the picture shown whenever your link is posted anywhere. Big difference in click-through.
- **Tagged Release:** create a release (e.g. v2.0) with short notes. A repo with a release looks maintained and trustworthy.
- **Pin the repo** on your GitHub profile (profile → “Customize your pins”).
- **Enable Discussions** (Settings → Features) so people have a place to ask questions.
- **README polish:** demo clip up top, a couple of screenshots, clear quick-start.

4.6 Other channels (optional, later)

- **YouTube:** a 2–3 minute screen-recorded demo with a voiceover. The local-AI crowd loves video; you can link it from every post and the README. High effort, high payoff.
- **X/Twitter + Mastodon:** a short post with the demo GIF + link, using tags like #LocalLLaMA, #LLM, #opensource. (Appendix A5.) Mastodon’s fediverse has an active self-hosting/AI crowd.
- **dev.to / Hashnode:** a “How I built a fully-local AI assistant” article — good for long-tail search traffic. (Outline in Appendix A7.)
- **Discord communities:** many local-AI Discords have a #projects/#show-and-tell channel. Share once, politely, with the demo — never spam multiple channels.
- **Product Hunt:** more for polished products than dev tools; optional, and best with a coordinated launch day.

5. Engagement playbook — handling the comments

The post is half the job; **how you respond is the other half**. Be present for the first couple of hours and answer everyone. Below are the questions you *will* get, with honest answers ready.

- **“Why not just use [ChatGPT / Claude / Ollama / Open WebUI]?”** → “Different goal — ELI is fully local and *acts on your machine* (desktop control, vision, voice, persistent memory), not just a chat UI. The trade is a smaller local model vs a frontier cloud one.”

- **“Why source-available and not open-source?”** → “I wanted the source open to read, run, and learn from, while keeping stewardship of the project. You can use and modify it freely; you just can’t redistribute it. Happy to hear arguments.”
- **“Isn’t this just a wrapper around llama.cpp?”** → “llama.cpp is the inference engine. ELI is the ~140k lines around it — routing, a 12-stage pipeline, 15 agents, memory, desktop/voice/vision integration, self-maintenance. The model is the brain; ELI is everything that makes it *do* things.”
- **“What model / how fast / how much VRAM?”** → “Model-agnostic GGUF; I run [YOUR MODEL]. Runs on 8 GB, auto-tunes layers/batch/context to your VRAM, scales to multi-GPU. Speed depends on your hardware.”
- **“Does it phone home?”** → “No. Offline by default, enforced at the socket — a process-wide guard fail-closes outbound connections. The few online actions (web search, news) are explicit and individually gated.”
- **“Another Jarvis clone?”** → “Fair skepticism. The difference is it actually does the things end-to-end and runs fully local; the honest caveat is the local-model ceiling. Try it and tell me where it falls short.”

Golden rules: thank critics (they’re giving you free QA), never get defensive, never argue to ‘win’, and **never ask for stars or upvotes**. A calm, honest author is the best advertisement.

6. The demo clip — your single most important asset

“It controls your computer and remembers you” is only believable if people **see it**. A short clip at the top of the README and in every post will do more than any paragraph.

- **Tools:** OBS Studio (free, all platforms) or your OS screen recorder to capture; then either keep it as a short MP4 or convert to GIF with ffmpeg (or a tool like Peek on Linux).
- **Length:** ~45–60 seconds, silent (or with a calm voiceover for a YouTube version).
- **Shot list (6–7 quick actions):**
 1. Say or type “open Firefox” → it opens.
 2. “what’s on my screen?” → it describes the screen.
 3. “play [a song] on Spotify” → it plays.
 4. Drag in a PDF → “summarize this” → it summarises.
 5. “remember that I prefer X” ... then “what do you know about me?” → it recalls.
 6. Turn your network **off** → ask something → it still answers (proves it’s local).
 7. End on the GUI or a clean, readable response.
- **Where it goes:** embed at the very top of the README (an MP4 can be uploaded into a GitHub issue/comment to get a hosted URL, or commit a GIF). Link the same clip in every post.

A good 60-second clip is worth more than everything else in this document combined.

7. Metrics and realistic expectations

- **What to track:** GitHub **Insights** → **Traffic** (views, unique visitors, referring sites), stars over time, and the upvotes/comments on each post. Referrers tell you which channel actually worked.
 - **Realistic numbers for a solo launch week:** hundreds–low-thousands of visitors, tens–low-hundreds of stars, a dozen real comments, a few people who try it. That’s a genuine success.
 - **Don’t tie your self-worth to the number next to a heart icon.** You build because you build; the value of ELI doesn’t move with a star count. Use the metrics to learn which channel to lean into, nothing more.
-

8. Do’s and don’ts (anti-spam survival guide)

Do: - Make one strong, honest post per channel, spaced days apart. - Be present and reply to everyone early. - Lead with privacy/local + a demo, and state limitations plainly. - Iterate the README from the

feedback you get.

Don't: - Blast the same link to many subs/sites on the same day. - Ask for upvotes, stars, or sponsorships (rules + it backfires). - Argue with critics or get defensive about the license. - Repost immediately if something flops — wait, improve, try a different angle later. - Use multiple accounts to upvote yourself (a fast ban everywhere).

Appendix A — Copy-paste drafts

A1. GitHub “About” one-liner (repo description field)

A fully-local, private AI assistant — voice, vision, desktop control, and persistent memory. Runs entirely on your machine on llama.cpp/GGUF. Offline by default.

A2. r/LocalLLaMA post

Title (pick one): 1. I built a fully-local AI assistant that runs 100% on your own machine — voice, vision, desktop control, persistent memory (offline-enforced) 2. ELI: a solo, 140k-line local assistant on llama.cpp/GGUF that actually operates your computer — source available

Body:

What it is

ELI runs entirely on your own machine — the model, your data, all of it. No cloud, no account, no telemetry. It's offline by default and actually *enforces* that: a process-wide network guard fail-closes outbound connections at the socket layer, and online actions (web search, news) are explicit and individually gated.

Built on llama.cpp/GGUF and model-agnostic — drop in any local GGUF, it detects the chat template, sizes context to the model's real `n_ctx_train`, and auto-tunes GPU layers/batch/context to fit your VRAM (runs on 8 GB; scales to multi-GPU). I daily-drive it on [YOUR MODEL].

What it actually does — not just chat: - Operates the desktop — opens/closes/focuses apps, tiles windows, types, clicks, controls media (Spotify/YouTube), clipboard, screenshots - Sees your screen — local vision model + OCR: “what's on my screen?”, describe images, summarize PDFs/CSVs - Voice — your own wake word, dictation (faster-whisper), TTS (Piper) - Optional webcam gaze control — click where your eyes rest - Persistent memory — SQLite + a FAISS vector index + a knowledge graph; builds a private profile of you across sessions; learns routines and offers to automate them - Coding agent — plan -> verify -> repair, fixes bugs in files, examines a codebase - Self-maintenance — logs its own failures, can patch its own source (with rollback), and can LoRA fine-tune the model on your own conversation history - Scheduling — alarms/timers/overnight jobs, proactive briefings

~140k lines of Python: a 12-stage reasoning pipeline, 14 specialist agents on a DAG orchestrator, a desktop GUI, a terminal interface, and an optional local-network web view for your phone. Cross-platform (Linux/CUDA, macOS/Metal, Windows).

The honest part

I'm one person and this is a big, sprawling project — ambitious, not perfectly polished. The real ceiling is the local model: a 7–8B on consumer hardware is nowhere near a frontier cloud model, and no amount of scaffolding fully hides that. The trade is deliberate — privacy, ownership, and offline operation in exchange for raw power — and you can swap in better open models as they land.

It's source-available, not OSI open-source: you can read, run, and modify it freely; you just can't redistribute it. I wanted the code open to learn from while keeping stewardship of the project.

Repo: https://github.com/ShadowESC95/ELI_v2.0

It's a solo build and I'd genuinely welcome feedback, criticism, and to hear whether it's useful to anyone but me.

A3. Hacker News "Show HN"

Title: Show HN: ELI - a fully-local AI assistant that controls your desktop (llama.cpp)

First comment (post immediately after submitting the repo URL):

ELI runs entirely on your own machine — model, data, and all processing local. No cloud, no account, no telemetry. It's offline by default and enforces it: a process-wide network guard fail-closes outbound connections at the socket layer; the few online actions (web search, news) are explicit and individually gated.

Built on llama.cpp/GGUF and model-agnostic — drop in any GGUF, it reads the chat template, sizes context to the model's real `n_ctx_train`, and auto-tunes GPU layers/batch/context to fit available VRAM (runs on 8 GB, scales to multi-GPU). I run it on [YOUR MODEL].

Beyond chat it acts on the machine: desktop control (apps, windows, input, media, clipboard, screenshots), screen reading via a local vision model + OCR, voice in/out (faster-whisper + Piper) with a configurable wake word, optional webcam gaze control, persistent memory (SQLite + FAISS + a knowledge graph) that builds a profile of you across sessions, a coding agent (plan -> verify -> repair), self-patching of its own source with rollback, LoRA fine-tuning on your own conversation history, and scheduled/overnight jobs.

~140k lines of Python: a 12-stage pipeline, 15 agents on a DAG orchestrator, a desktop GUI, a terminal interface, and an optional local-network web view.

Honest caveats: I'm one person; it's large and not perfectly polished. The ceiling is the local model — a 7-8B on consumer hardware is well short of a frontier cloud model, and the scaffolding doesn't hide that. The bet is that privacy, ownership, and offline operation are worth it, and that you can swap in stronger open models as they arrive.

License: source-available (PolyForm Internal Use), not OSI open-source — read, run, and modify, but not redistribute. Happy to discuss the choice; I wanted the source open to learn from while keeping stewardship.

Feedback and criticism welcome.

A4. r/selfhosted variant (reframed — privacy/ownership angle)

Title: A fully self-hosted, offline AI assistant — runs entirely on your own hardware, no cloud, no telemetry

Body:

I wanted an assistant I fully own — no cloud, no account, nothing leaving the machine — so I built ELI. It runs entirely on your own hardware on llama.cpp/GGUF and is offline by default, enforced at the network-socket level (a process-wide guard fail-closes outbound connections; the few online features like web search are explicit and individually gated).

It does more than chat: controls the desktop (apps, windows, media, clipboard, screenshots), reads your screen (local vision + OCR), voice in/out, persistent local memory that builds a private profile of you across sessions, a coding agent, and scheduled/overnight jobs. Desktop GUI plus an optional local-network web view so you can reach it from your phone — still all on your LAN, nothing external.

~140k lines of Python, cross-platform (Linux/CUDA, macOS/Metal, Windows). Honest caveat: a local model on consumer hardware is less capable than a cloud model — the trade is privacy and ownership. Source-available (you can run and modify it, not redistribute).

Repo: https://github.com/ShadowESC95/ELI_v2.0 — feedback welcome.

A5. X/Twitter + Mastodon short post

Built ELI: a fully-local AI assistant that runs 100% on your own machine — voice, vision, desktop control, and persistent memory, on llama.cpp/GGUF. No cloud, no telemetry, offline by default. Solo project, source available. [link] #LocalLLaMA #LLM #opensource

(Attach the demo GIF — it massively increases engagement.)

A6. Reusable one-paragraph blurb (for anywhere)

ELI is a fully-local AI assistant that runs entirely on your own machine — model, data, and all processing on your hardware, offline by default. Beyond chat it operates your desktop, sees your screen, talks and listens, and builds a private memory of you over time. Built on llama.cpp/GGUF and model-agnostic. Source-available; ~140k lines of Python.

A7. dev.to / blog article outline (“How I built a fully-local AI assistant”)

1. The itch: why I wanted a private, local assistant (the cloud trade-offs).
 2. Architecture overview: the pipeline, the agents, the memory stack (diagram).
 3. Making a local model act: routing + the executor + grounding.
 4. Keeping it honest: the no-fake-actions / grounded-introspection design.
 5. Offline-by-default, enforced at the socket: how and why.
 6. The hard parts / what I’d do differently (this section earns the most respect).
 7. Try it: repo link + quick start.
-

Appendix B — The 60-second pre-flight checklist (before you click “post”)

- ☐ Demo clip is at the top of the README and ready to attach to the post.
- ☐ Social-preview image is set (Settings → Social preview).
- ☐ README first screen is clean; quick-start works from a fresh clone.
- ☐ Repo topics + license + Sponsor button are in place.
- ☐ [YOUR MODEL] placeholder is replaced in the drafts.
- ☐ Account is warmed up; you’ve read the subreddit’s rules and picked flair.
- ☐ It’s a weekday morning (US ET) and you have ~2 hours free to reply to comments.
- ☐ You’re posting to **one** place today, not five.

Do these, post the one draft, and stay to talk to people. That’s the whole game.